Oops Assignment - #1

1. What is an object in C++?
A) A function of a class
B) An instance of the class
C) A data type of the class
D) More than one of the above
E) None of the above

2. Where is memory allocated for objects in C++?
A) RAM
B) ROM
C) Cache
D) More than one of the above
E) None of the above

3. What determines when a variable comes into existence in memory?
A) Data type
B) Storage class
C) Scope
D) More than one of the above
E) None of the above

4. Which of the following shows the correct syntax for a constructor?
A) ()class_name
B) -class_name
C) class_name()
D) More than one of the above
E) None of the above

5. To access data members of a class, which of the following is used?
A) Dot operator
B) Arrow operator
C) Dot or arrow operator, as required
D) More than one of the above
E) None of the above

6. How many access specifiers are there in C++?
A) 1
B) 2
C) 3
D) More than one of the above
E) None of the above

7. How many instances of an abstract class can be created?
A) 0
B) 1
C) 4
D) More than one of the above
E) None of the above

8. How many types of constructors are there in C++?
A) 1
B) 2
C) 3
D) More than one of the above
E) None of the above

9. How many types of polymorphism exist in C++?
A) 1
B) 2
C) 3
D) More than one of the above
E) None of the above

10. Under which pillar of OOPS does the base class and derived class relationship come?
A) Polymorphism
B) Inheritance
C) Encapsulation
D) More than one of the above
E) None of the above

11. Virtual functions in C++ should be declared as:
A) Public
B) Private
C) Protected
D) More than one of the above
E) None of the above

12. Where should virtual functions be defined?
A) Derived class
B) Base class
C) Both base and derived class
D) More than one of the above
E) None of the above

13. How many catch blocks can be used with a single try block?
A) 1
B) 2
C) As many as required
D) More than one of the above
E) None of the above

14. Identify the scope resolution operator in C++:
A) :
B) ::
C) ?:
D) More than one of the above
E) None of the above

Subscribe Infeepedia youtube channel for computer science competitive exams
Download Infeepedia app and call or wapp on 8004391758

15. What is cout in C++?
A) Operator
B) Object
C) Function
D) More than one of the above
E) None of the above

16. What is the implicit return type of a constructor in C++?
A) No return type
B) A class object in which it is defined
C) void
D) More than one of the above
E) None of the above

17. How many parameters does a default constructor take?
A) 0
B) 1
C) 2
D) More than one of the above
E) None of the above

18. What type of inheritance does single-level inheritance support?
A) Compile-time inheritance
B) Runtime inheritance
C) Multiple inheritances
D) More than one of the above
E) None of the above

19. When is an object created using the new keyword?
A) At runtime
B) At compile time
C) Depends on the code
D) More than one of the above
E) None of the above

20. Which of the following functions cannot be inherited from a base class?
A) Constructor
B) Destructor
C) Static functions
D) More than one of the above
E) None of the above

21. Which of the following is NOT a type of inheritance?
A) Multiple inheritance
B) Multilevel inheritance
C) Distributed inheritance
D) More than one of the above
E) None of the above

22. Which of the following is NOT an OOP concept?
A) Inheritance
B) Compilation
C) Polymorphism
D) More than one of the above
E) None of the above

23. Which type of inheritance is demonstrated by a class inheriting from two or more base classes?
A) Single inheritance
B) Multilevel inheritance
C) Hybrid inheritance
D) Multiple inheritance
E) None of the above

24. Who is credited with developing Object-Oriented Programming (OOP)?
A) Dennis Ritchie
B) Charles Babbage
C) Alan Kay
D) More than one of the above
E) None of the above

25. What is the primary purpose of a virtual function in C++?
A) Achieve function code polymorphism
B) Achieve compile-time polymorphism
C) Achieve runtime polymorphism
D) More than one of the above
E) None of the above

26. Why is reusability considered a key feature of Object-Oriented Programming?
A) Reduces compilation time
B) Decreases testing time
C) Lowers maintenance cost
D) More than one of the above
E) None of the above

27. Can a constructor function be declared as const in C++?
A) True
B) False
C) Depends on implementation
D) More than one of the above
E) None of the above

28. What is another name for a generic class in C++?
A) Template class
B) Final class
C) Abstract class
D) More than one of the above
E) None of the above

29. By default, the fields in a structure in C++ are:
A) Public
B) Private
C) Protected
D) More than one of the above
E) None of the above

30. Which of the following is NOT a valid type of constructor in C++?
A) Copy constructor
B) Friend constructor
C) Parameterized constructor
D) More than one of the above
E) None of the above

31. For which type of class can an instance not be created?
A) Anonymous class
B) Parent class
C) Abstract class
D) More than one of the above
E) None of the above

32. Which operator can be overloaded to demonstrate polymorphism?
A) ||
B) <<
C) &&
D) More than one of the above
E) None of the above

33. Which of the following describes the overloading of functions?
A) Ad-hoc polymorphism
B) Runtime polymorphism
C) Compile-time polymorphism
D) More than one of the above
E) None of the above

34. Which of the following is not a member of the class?
A) Friend function
B) Static function
C) Virtual function
D) More than one of the above
E) None of the above

35. What feature does function overriding demonstrate?
A) Abstraction
B) Encapsulation
C) Polymorphism
D) More than one of the above
E) None of the above

36. Which of the following is used to free memory allocated for an object in C++?
A) free()
B) delete
C) unallocate()
D) More than one of the above
E) None of the above

37. Which of the following is used to implement late binding?
A) Friend function
B) Static function
C) Virtual function
D) More than one of the above
E) None of the above

38. Which operator can be overloaded using a friend function?
A) +
B) *
C) =
D) More than one of the above
E) None of the above

39. Can an object be passed as a function?
A) Yes
B) No
C) Depends on the implementation
D) More than one of the above
E) None of the above

40. What is another name for function overloading?
A) Ad-hoc polymorphism
B) Transient polymorphism
C) Virtual polymorphism
D) More than one of the above
E) None of the above

41. By default, are the data members and member functions of a class private?
A) True
B) False
C) Depends on the code
D) More than one of the above
E) None of the above

42. Which of the following can be used for dynamic memory allocation?
A) calloc()
B) malloc()
C) Both A and B
D) More than one of the above
E) None of the above

Subscribe Infeepedia youtube channel for computer science competitive exams
Download Infeepedia app and call or wapp on 8004391758

43. How do encapsulation and abstraction differ?
A) Hiding and Binding
B) Binding and Hiding
C) Hiding and Hiding
D) More than one of the above
E) None of the above

44. Identify the abstract data type from the following.
A) double
B) int
C) class
D) More than one of the above
E) None of the above

45. Which feature allows one object to interact with another object?
A) Message passing
B) Message reading
C) Data binding
D) More than one of the above
E) None of the above

46. Which feature can be implemented using encapsulation?
A) Abstraction
B) Polymorphism
C) Inheritance
D) More than one of the above
E) None of the above

47. Which feature reduces the need for nested classes?
A) Encapsulation
B) Inheritance
C) Abstraction
D) More than one of the above
E) None of the above

48. Which of the following is not a valid constructor type?
A) Friend constructor
B) Default constructor
C) Parameterized constructor
D) More than one of the above
E) None of the above

49. Which operator cannot be overloaded?
A) ?:
B) . (dot operator)
C) >>
D) More than one of the above
E) None of the above

50. Which of the following cannot be used with the term "virtual"?
A) Constructor
B) Destructor
C) Member function
D) More than one of the above
E) None of the above

51. Which of the following is not a property of an object?
A) Name
B) Attributes
C) Identity
D) More than one of the above
E) None of the above

52. Which two features of object-oriented programming are the same?
A. Abstraction and Polymorphism features are the same
B. Inheritance and Encapsulation features are the same
C. Encapsulation and Abstraction features are the same
D. More than one of the above
E. None of the above

53. Which header file is required by the C++ programming language to use the OOPS concept?
A. stdio.h
B. iostream.h
C. stdlib.h
D. More than one of the above
E. None of the above

54. Which of the following definition is incorrect for polymorphism?
A. Polymorphism helps in redefining the same functionality
B. Polymorphism concept is the feature of object-oriented programming (OOP)
C. It always increases the overhead of function definition
D. Ease in the readability of the program
E. None of the above

55. Which among the following cannot be used for the concept of polymorphism?
A. Static member function
B. Constructor Overloading
C. Member function overloading
D. More than one of the above
E. None of the above

56. Which function best describes the concept of polymorphism in programming languages?
   A. Class member function
   B. Virtual function
   C. Inline function
   D. More than one of the above
   E. None of the above

57. Which member function is assumed to call first when there is a case of using function overloading or abstract class?
   A. Global function
   B. Local function
   C. Function that best matches the call signature
   D. More than one of the above
   E. None of the above

58. Which of the following language uses the classes but not the polymorphism concept?
   A. Procedure Oriented Language
   B. Object-Based Language
   C. Class-Based Language
   D. More than one of the above
   E. None of the above

59. Which of the following OOP concept is not true for the C++ programming language?
   A. A class must have member functions
   B. C++ Program can be easily written without the use of classes
   C. At least one instance should be declared within the C++ program
   D. More than one of the above
   E. None of the above

60. What is the extra feature in classes which was not in the structures?
   A. Member functions
   B. Data members
   C. Public access specifier
   D. More than one of the above
   E. None of the above

61. Which of the following feature is also known as run-time binding or late binding?
   A. Dynamic typing
   B. Dynamic loading
   C. Dynamic binding
   D. More than one of the above
   E. None of the above

62. Which among the following is not a member of the class?
   A. Virtual function
   B. const function
   C. Static function
   D. Friend function
   E. None of the above

63. Which operator overloads using the friend function?
   A. *
   B. ( )
   C. ->
   D. More than one of the above
   E. None of the above

64. Which of the following OOP concept binds the code and data together and keeps them secure from the outside world?
   A. Polymorphism
   B. Encapsulation
   C. Abstraction
   D. More than one of the above
   E. None of the above

65. Which member of the superclass is never accessible to the subclass?
   A. Public member
   B. Protected member
   C. Private member
   D. More than one of the above
   E. None of the above

66. What is the size of a class?
   A. Sum of the size of all inherited variables along with the variables of the same class
   B. The size of the class is the largest size of the variable of the same class
   C. Classes in the programming languages do not have any size
   D. More than one of the above
   E. None of the above

67. Which class cannot create its instance?
   A. Parent class
   B. Nested class
   C. Anonymous class
   D. More than one of the above
   E. None of the above

68. Which of the following variable violates the definition of encapsulation?
- A. Array variables
- B. Local variables
- C. Global variables
- D. More than one of the above
- E. None of the above

69. Encapsulation is_____?
- A. A technique of combining more than one member function into a single unit
- B. A mechanism of combining more than one data member into a single unit
- C. A mechanism of combining more than one data member and member functions that implement on those data members into a single unit
- D. More than one of the above
- E. None of the above

70. Which of the following statement of a program is not right?
- A. class teacher { }; teacher s[5];
- B. class teacher { } s;
- C. class teacher { }; teacher s;
- D. class teacher { } s[];
- E. None of the above

71. Which of the following syntax is incorrect for the class definition?
- A. student class { };
- B. class student { student(int a) {} };
- C. class teacher { public: teacher(int a) { } };
- D. More than one of the above
- E. None of the above

72. The object cannot be_____?
- A. Passed by copy
- B. Passed as function
- C. Passed by value
- D. More than one of the above
- E. None of the above

73. Which among the following feature does not come under the concept of OOPS?
- A. Data binding
- B. Data hiding
- C. Platform independent
- D. Message passing
- E. None of the above

74. Which of the following feature may be broken if the user does not use the classes in the code?
- A. Object must be used violated
- B. Only the encapsulation concept is violated
- C. Inheritance cannot be implemented
- D. More than one of the above
- E. None of the above

75. Which of the following feature interacts one object with another object?
- A. Message reading
- B. Message passing
- C. Data transfer
- D. Data binding
- E. None of the above

76. Which definition best defines the concept of abstraction?
- A. Hides the important data
- B. Hides the implementation
- C. Shows the important data
- D. More than one of the above
- E. None of the above

77. The combination of abstraction of the data and code is viewed in_____.
- A. Object
- B. Class
- C. Interfaces
- D. More than one of the above
- E. None of the above

78. The principle of abstraction_____.
- A. Is used to achieve OOPS
- B. Is used to avoid duplication
- C. Use abstraction at its minimum
- D. More than one of the above
- E. None of the above

79. Which among the following concept is correct if a user uses the concept of encapsulation in a code?
- A. The modification of the code can be additional overhead
- B. Member functions can be used for modifying the data type of data members
- C. The data type of the data member can be easily modified without modifying any other code
- D. More than one of the above
- E. None of the above

80. Using the concept of encapsulation, security of the data is _____.

A. Ensured to some extent
B. Purely ensured
C. Not ensured
D. More than one of the above
E. None of the above

81. Consider the following program and select the right option from the given options:

```
class marksofstudent {
   int subjectmarks;
   public : int* fun() {
      return &subjectmarks;
   }
};
int main() {
   marksofstudent s;
   int *ptr = s.fun();
   return 0;
}
```

A. The above program violates the feature of encapsulation
B. The above program may result in undesirable conditions
C. The above program will generate an error
D. The above program is good to go
E. None of the above

82. The name of the default access specifier for the member functions or data members in the C++ programming language is_____.

A. Private access specifier
B. Public access specifier
C. Protected access specifier
D. More than one of the above
E. None of the above

83. Which of the following option best illustrates a friend class?

A. This class can access and manipulate all the private members of that class which connects to a friend
B. Friend class can only access and manipulate the protected data members of that class that connects to a friend
C. Friend class can't access any data member of another class but can use its methods
D. More than one of the above
E. None of the above

84. Which of the following definition best describes the concept of polymorphism?

A. It is the ability to process many messages and data in one way
B. It is the ability to process undefined messages or data in at least one way
C. It is the ability to process the message or data in more than one form
D. More than one of the above
E. None of the above

85. Which class or set of classes can describe the concept of polymorphism in the following code?

```
class student_details
 { public: int marksofstudent;
virtual int calculate_grade() = 0;
};
 class topper : public student_details
{ public: int calculate_grade()
 { return 15; }
};
class average : public student_details
{ public: int calculate_grade()
{ return 30; }
 };
```

A. Only the student_details class can show the concept of polymorphism
B. The class failed should also inherit class student_details for this code to work for polymorphism
C. The student_details, topper, and average classes together can show the concept of polymorphism
D. More than one of the above
E. None of the above

86. Inline functions are useful when:

A. Function is large with many nested loops.
B. Function has many static variables.
C. Function is small and we want to avoid function call overhead.
D. More than one of the above.
E. None of the above.

87. What is the output of the following code?

```
#include<iostream>
using namespace std;
int x = 1;
void fun()
{
   int x = 2;
   {
      int x = 3;
      cout << ::x << endl;
   } }
int main()
{
   fun();
   return 0;
}
```

A. 1
B. 2
C. 3
D. More than one of the above.
E. None of the above.

88. How can we make a C++ class such that objects of it can only be created using the new operator?
A. Not possible.
B. By making the destructor private.
C. By making the constructor private.
D. More than one of the above.
E. None of the above.

89. Would the destructor be called in the following program?

```
#include <iostream>
#include <vector>
using namespace std;
class a {
public:
   ~a() { cout << "destroy"; }
};
int main() {
   vector<a*> *v1 = new vector<a*>;
   vector<a> *v2 = new vector<a>;
   return 0;
}
```

A. v1
B. v2
C. Both v1 and v2.
D. More than one of the above.
E. None of the above.

90. What is the output of the following code?

```
#include<iostream>
using namespace std;
int x[100];
int main()
{
   cout << x[99] << endl;
}
```

A. Unpredictable.
B. Runtime error.
C. 0.
D. More than one of the above.
E. None of the above.

91. A member function can always access the data in _____.
A. The class of which it is a member.
B. The object of which it is a member.
C. The public part of its class.
D. More than one of the above.
E. None of the above.

92. Which of the following is not correct for a virtual function in C++?
A. Must be declared in the public section of the class.
B. Virtual function can be static.
C. Virtual function should be accessed using pointers.
D. More than one of the above.
E. None of the above.

93. In C++, polymorphism requires:
A. Inheritance only.
B. Virtual functions only.
C. References only.
D. More than one of the above.
E. None of the above.

94. Method overriding can be prevented by using final as a modifier at _____.
A. The start of the class.
B. The start of the method declaration.
C. The start of the derived class.
D. More than one of the above.
E. None of the above.

95. Which of the following in Object-Oriented Programming is supported by Function Overloading and Default Arguments features of C++?
A. Inheritance
B. Polymorphism
C. Encapsulation
D. More than one of the above
E. None of the above

96.Which of the following overloaded functions are NOT allowed in C++?

1. Function declarations that differ only in the return type.

```
int fun(int x, int y);
void fun(int x, int y);
```

2. Functions that differ only by the static keyword in the return type.

```
int fun(int x, int y);
static int fun(int x, int y);
```

3. Parameter declarations that differ only in a pointer * versus an array [].

```
int fun(int *ptr, int n);
int fun(int ptr[], int n);
```

4. Two parameter declarations that differ only in their default arguments.

```
int fun(int x, int y);
int fun(int x, int y = 10);
```

A. All of the above
B. All except 2
C. All except 2 and 4
D. More than one of the above
E. None of the above

97. What is the output of the following program?

```
#include <iostream>
using namespace std;
int fun(int = 0, int = 0);
int main() {
   cout << fun(5);
   return 0;
}
int fun(int x, int y) { return (x + y); }
```

A. Compiler Error
B. 5
C. 0
D. More than one of the above
E. None of the above

98. Predict the output of the following C++ program:

```
#include <iostream>
using namespace std;
 class Test {
protected:
   int x;
public:
   Test(int i) : x(i) {}
   void fun() const { cout << "fun() const " << endl; }
   void fun() { cout << "fun() " << endl; }
};
 int main() {
   Test t1(10);
   const Test t2(20);
   t1.fun();
   t2.fun();
   return 0;
}
```

A. Compiler Error
B. fun() fun() const
C. fun() const fun() const
D. fun() fun()
E. None of the above

100. Which of the following is NOT correct for virtual functions in C++?

A. Must be declared in the public section of the class.
B. Virtual functions can be static.
C. Virtual functions should be accessed using pointers.
D. Virtual functions are defined in the base class.
E. None of the above

Answer With Explanation

1. Answer: B) An instance of the class

Explanation: An object in C++ is a runtime entity that represents an instance of a class. It is created based on the blueprint defined by the class and holds specific data and methods associated with that class. Objects allow the program to interact with the class's attributes and functions. While the class defines the structure, the object is the tangible entity that uses this structure.

2. Answer: A) RAM

Explanation: Memory for objects in C++ is allocated in RAM. This memory allocation can happen on the stack or heap, depending on how the object is created. Objects created using the new keyword are allocated on the heap, while local objects are stored in the stack. RAM provides the working memory necessary for executing programs and storing runtime data.

3. Answer: D) More than one of the above

Explanation: The existence of a variable in memory depends on multiple factors:

- **Data type: Determines the size and type of data the variable can store.**
- **Storage class: Specifies the variable's lifetime and visibility (e.g., auto, static, extern).**
- **Scope: Defines the region in the code where the variable is accessible.**

Together, these factors govern the variable's lifecycle, allocation, and accessibility.

4. Answer: C) class_name()

Explanation: A constructor in C++ is a special member function with the same name as the class. It is automatically invoked when an object of the class is created. The correct syntax for a constructor is class_name(). Constructors do not have a return type, and they are used to initialize the object's data members.

5. Answer: D) More than one of the above

Explanation: The dot operator (.) is used to access members of an object directly. The arrow operator (->) is used to access members through a pointer to an object.

Both operators are necessary in different contexts, making "More than one of the above" the correct answer.

6. Answer: C) 3

Explanation: C++ provides three access specifiers:

Private: Members are accessible only within the class.

Protected: Members are accessible within the class and its derived classes.

Public: Members are accessible from anywhere in the program.

These specifiers control the visibility and accessibility of class members.

7. Answer: A) 0

Explanation: An abstract class cannot be instantiated directly. It serves as a blueprint for other classes and may contain pure virtual functions. Derived classes must implement these functions to create concrete objects. Abstract classes are used to define common interfaces.

8. Answer: C) 3

Explanation:There are three types of constructors in C++:

Default constructor: Takes no parameters and initializes objects with default values.

Parameterized constructor: Takes arguments to initialize objects with specific values.

Copy constructor: Initializes an object by copying data from another object.

9. Answer: B) 2

Explanation: C++ supports two types of polymorphism:

Compile-time polymorphism: Achieved using function overloading and operator overloading.

Runtime polymorphism: Achieved using virtual functions and inheritance.

Subscribe Infeepedia youtube channel for computer science competitive exams

Download Infeepedia app and call or wapp on 8004391758

Polymorphism allows a single interface to represent different underlying forms (data types or functions).

10. Answer: B) Inheritance
Explanation: Inheritance allows a derived class to inherit properties and behaviors from a base class. This promotes code reuse and establishes a parent-child relationship between classes. It is a fundamental pillar of Object-Oriented Programming (OOP).

11. Answer: A) Public
Explanation: Virtual functions are usually declared as public in the base class to ensure they are accessible and can be overridden in derived classes. Declaring a virtual function enables runtime polymorphism, allowing the appropriate function to be called based on the object's type during runtime.

12. Answer: B) Base class
Explanation: Virtual functions are declared in the base class to allow overriding in derived classes. The base class provides a default implementation, but derived classes can override it to provide specific functionality. If not defined in the base class, the function cannot be virtual.

13. Answer: C) As many as required
Explanation: C++ allows multiple catch blocks for a single try block to handle different types of exceptions. Each catch block can handle a specific exception type, enabling fine-grained error handling. This flexibility ensures robust exception management.

14. Answer: B) ::
Explanation: The scope resolution operator (::) is used in C++ to access a global variable when a local variable has the same name, or to define a function outside its class. It is also used to access static members of a class and to specify namespaces.

15. Answer: B) Object
Explanation: cout is an object of the ostream class in C++ and is used to output data to the standard output (usually the console). It is part of the iostream library and is often used in conjunction with the insertion operator (<<) to display information.

16. Answer: A) No return type
Explanation: Constructors in C++ do not have a return type, not even void. They are special member functions automatically called to initialize objects of a class. They cannot return values, and their primary purpose is to set up the initial state of an object.

17. Answer: A) 0
Explanation:A default constructor does not take any parameters. It initializes class members with default values. If no constructor is explicitly defined, the compiler provides a default constructor automatically.

18. Answer: B) Runtime inheritance
Explanation: Single-level inheritance allows a derived class to inherit from a single base class. The inheritance relationship is established during runtime, enabling the derived class to use and override the base class's members.

19. Answer: A) At runtime
Explanation: Objects created using the new keyword are dynamically allocated on the heap at runtime. This allows the programmer to control the object's lifetime and memory usage. Such objects must be explicitly deleted to avoid memory leaks.

20. Answer: D) More than one of the above
Explanation: Constructors: Not inherited because they are specific to the class being instantiated.

Subscribe Infeepedia youtube channel for computer science competitive exams
Download Infeepedia app and call or wapp on 8004391758

Destructors: Not inherited but are called in reverse order of construction during object destruction.
Static functions: Not inherited but can be accessed using the class name.
These functions are not directly inherited as they do not fit the inheritance model.

21. Answer: C) Distributed inheritance
Explanation: C++ supports five types of inheritance: single, multiple, multilevel, hierarchical, and hybrid. Distributed inheritance is not a valid type in C++. It might be a concept in distributed systems but does not apply to C++ inheritance.

22. Answer: B) Compilation
Explanation: Compilation is a process of converting source code into machine code and is not related to Object-Oriented Programming (OOP). OOP concepts include inheritance, polymorphism, encapsulation, and abstraction.

23. Answer: D) Multiple inheritance
Explanation: In multiple inheritance, a derived class inherits from two or more base classes. This allows the derived class to combine features from all the base classes. While powerful, multiple inheritance can lead to ambiguity, especially when two base classes have members with the same name.

24. Answer: C) Alan Kay
Explanation: Alan Kay is considered one of the pioneers of Object-Oriented Programming. He developed the concept while working on the Smalltalk programming language. His work laid the foundation for modern OOP languages like C++, Java, and Python.

25. Answer: C) Achieve runtime polymorphism
Explanation: Virtual functions enable runtime polymorphism by allowing the program to decide at runtime which function to call based on the object's type. This is achieved through the use of a virtual table (vtable). It allows dynamic binding of functions.

26. Answer: D) More than one of the above
Explanation: Reusability in OOP, achieved through inheritance and modular design, reduces development time, decreases testing time, and lowers maintenance costs. It allows code to be reused across different projects or modules, improving efficiency and reliability.

27. Answer: B) False
Explanation: Constructors cannot be declared as const in C++ because they are meant to initialize objects, which inherently involves modifying the object's state. Declaring them as const would contradict their purpose.

28. Answer: A) Template class
Explanation: A generic class in C++ is called a template class. Templates allow the creation of classes and functions that work with any data type, promoting code reusability and flexibility. They are widely used in the Standard Template Library (STL).

29. Answer: A) Public
Explanation: In C++, the members of a struct are public by default, unlike a class, where members are private by default. This default access level allows straightforward access to structure members unless explicitly modified.

30. Answer: B) Friend constructor
Explanation: There is no such thing as a "friend constructor" in C++. The valid types of constructors are default constructors, parameterized constructors, and copy constructors. The friend keyword is used to grant access to private members, not to define constructors.

31. Answer: C) Abstract class
Explanation: An abstract class cannot be instantiated because it contains at least one pure virtual function. It acts as a

template for other classes and enforces derived classes to implement the pure virtual functions. For example, if a class Shape is abstract, you cannot create Shape obj; but must use derived classes like Circle or Rectangle to instantiate objects. Anonymous and parent classes can have instances, but abstract classes are designed to provide a base structure for inheritance.

32. Answer: B) <<
Explanation:
The << operator is widely overloaded in C++ for polymorphic behavior, especially in input/output operations. For example, std::cout << "Hello"; uses the overloaded << operator. Logical operators like || and && cannot be overloaded in C++. Overloading << allows developers to define custom behaviors for their classes when interacting with streams, making it a key example of polymorphism in operator overloading.

33. Answer: A) Ad-hoc polymorphism
Explanation: Function overloading is a form of ad-hoc polymorphism where multiple functions with the same name exist in a class but differ in their parameter lists. This is resolved at compile time. For example, in C++, you can define void display(int) and void display(double) in the same class. While runtime polymorphism involves virtual functions, ad-hoc polymorphism specifically refers to overloading.

34. Answer: A) Friend function
Explanation: A friend function is not a member of the class but has access to its private and protected members. Static and virtual functions, on the other hand, are members of the class. A friend function is declared using the friend keyword and can operate on the class's internal data without being a member. It is often used to provide external functions special access to a class's data.

35. Answer: C) Polymorphism
Explanation: Function overriding is an example of runtime polymorphism. It allows a derived class to provide a specific implementation of a function that is already defined in its base class. The base class function must be marked as virtual to enable overriding. This mechanism enables dynamic method dispatch, where the function that gets called is determined at runtime based on the object's type.

36. Answer: B) delete
Explanation: In C++, the delete operator is used to free memory allocated for objects created with the new keyword. For example, if int* ptr = new int;, then delete ptr; releases the memory. The free() function is used in C for memory allocated with malloc() or calloc() but is not used for objects in C++. There is no function named unallocate() in C++.

37. Answer: C) Virtual function
Explanation: Late binding refers to the process where the function to be called is determined at runtime. Virtual functions in C++ enable late binding. When a base class declares a function as virtual, derived classes can override it. The function call is resolved based on the type of object being pointed to, not the type of pointer, enabling polymorphism.

38. Answer: D) More than one of the above
Explanation: Operators like +, *, and = can be overloaded using friend functions. A friend function has special privileges to access private and protected members of a class, which is useful when overloading operators. For example, overloading the + operator for a Complex class allows adding two complex numbers using Complex c3 = c1 + c2;.

39. Answer: A) No
Explanation: Objects can be passed as arguments to functions in C++ but not as a function. you can pass an object of the class to a function. Objects can be passed by value, by reference, or by pointer.

40. Answer: A) Ad-hoc polymorphism
Explanation: Function overloading is also known as ad-hoc polymorphism because it allows multiple functions with the same name to coexist, differentiated by their parameter types or number of parameters. This is resolved during compile time, making it a form of compile-time polymorphism.

41. Answer: A) True
Explanation: In C++, the default access specifier for data members and member functions of a class is private. This means they are not accessible directly from outside the class. For example:
```
class Example {
   int x; // x is private by default
};
```
To make members public or protected, the corresponding access specifiers must be explicitly declared.

42. Answer: D) More than one of the above
Explanation: In C, dynamic memory allocation is done using malloc() and calloc(). While malloc() allocates memory without initializing it, calloc() allocates memory and initializes it to zero. Both functions are available in C++ as well, though new and delete are preferred for object-oriented programming.

43. Answer: B) Binding and Hiding
Explanation: Encapsulation focuses on binding data and functions together into a single unit (class), while abstraction is about hiding the implementation details and exposing only the essential features. For instance, a car's steering wheel provides abstraction, while encapsulation is the mechanism that binds the steering wheel and the car's internal mechanics.

44. Answer: C) class
Explanation: A class is considered an abstract data type (ADT) because it provides a way to define and manipulate data in a user-defined manner. Primitive types like int and double are not ADTs as they are predefined and lack user-defined operations or encapsulation.

45. Answer: A) Message passing
Explanation: Message passing enables objects to communicate with each other by invoking methods. For example, in object-oriented programming, calling object1.method(object2) is a form of message passing where object1 interacts with object2. This interaction forms the basis of object-oriented design.

46. Answer: A) Abstraction
Explanation: Encapsulation is the mechanism of bundling data and methods together. It provides the foundation for abstraction by hiding implementation details and exposing only the necessary functionality to the user. For example, getters and setters in a class encapsulate data while providing an abstract interface.

47. Answer: B) Inheritance
Explanation: Inheritance allows a class to reuse properties and methods of another class, reducing the need for complex nested class structures. For example, instead of nesting a class Engine inside a class Car, you can create an Engine class and have Car inherit its properties.

48. Answer: A) Friend constructor
Explanation: There is no such thing as a "friend constructor" in C++. Constructors cannot be declared as friend because they are specific to the class they belong to. Valid constructor types include default, parameterized, and copy constructors.

49. Answer: D) More than one of the above
Explanation: Operators like ?: (ternary operator) and . (dot operator) cannot be overloaded in C++. These operators are fundamental to the language and are handled directly by the compiler. Other operators like >> can be overloaded, for example, for input/output operations.

50. Answer: A) Constructor
Explanation: Constructors cannot be declared as virtual because they are called only once when an object is created.

However, destructors, member functions, and even classes can be declared as virtual to enable polymorphism and dynamic behavior in inheritance hierarchies.

51. Answer: A) Name
Explanation: An object has properties such as attributes (data members), identity (unique instance in memory), and behavior (methods). However, the name of an object is not its property but rather a reference or identifier used in the code. The properties are intrinsic to the object itself.

52. Answer: D. Encapsulation and Abstraction
Explanation: Encapsulation and abstraction are closely related OOP concepts.
Encapsulation hides the internal details of an object and binds its data and functions together within a class.
Abstraction simplifies complexity by showing only the necessary details and hiding the implementation.
Both aim to simplify program structure and enhance security, making them appear conceptually similar.

53. Answer: E. None of the above
Explanation: There is no specific header file required to implement OOP in C++.
- The headers like <iostream> are used for input/output operations, not OOP concepts.
- OOP features like classes and objects are built into the language itself, so you can use them without any specific header file.

54. Answer: C. It always increases the overhead of function definition
Explanation: Polymorphism, a core feature of OOP, does not inherently increase function definition overhead.
- Compile-time polymorphism (e.g., function overloading) enhances efficiency by resolving calls at compile time.
- Runtime polymorphism (e.g., virtual functions) can introduce slight overhead due to dynamic binding but offers flexibility.
  Thus, saying it "always increases overhead" is incorrect.

55. Answer: A. Static member function
Explanation: Static member functions are not bound to object instances; they belong to the class.
- Polymorphism, which relies on dynamic behavior, cannot apply to static member functions since they cannot be overridden or overloaded in the same way.

56. Answer: B. Virtual function
Explanation: Virtual functions in C++ allow achieving runtime polymorphism.
- They enable dynamic dispatch, where the function that gets called is determined by the type of the object at runtime rather than compile time.
- This behavior contrasts with regular or inline functions, which are resolved statically.

57. Answer: D. Function that best matches the call signature
Explanation: When multiple functions are overloaded or abstract classes are used, the compiler matches the most specific function call based on arguments.
- This is known as function resolution or function matching during compile time.
- The function that best matches the call signature (highest priority) is invoked first.

58. Answer: B. Object-Based Language
Explanation: Object-based languages, such as JavaScript (in earlier versions), support the use of classes and objects but lack core OOP features like polymorphism and inheritance.
These languages focus on encapsulation and modular programming without full-fledged OOP support.

59. Answer: D. More than one of the above
Explanation: A class in C++ does not need to have member functions. A C++ program can be written without using

classes, as procedural programming is also supported. Therefore, both options 1 and 3 are incorrect, making "More than one of the above" the correct choice.

60. Answer: A. Member functions
Explanation: In C++, structures can have access specifiers and data members, just like classes.
- However, classes provide the additional ability to define member functions, which encapsulate behaviors along with data.
- This makes classes more versatile than structures.

61. Answer: C. Dynamic binding
Explanation: Dynamic binding refers to the process where a function call is resolved at runtime.
- It allows programs to determine the appropriate method to execute based on the object type.
- This is achieved using virtual functions in C++.

62. Answer: D. Friend function
Explanation: A friend function is not a member of the class, but it has special access to private and protected members of the class.
- Unlike virtual, const, or static functions, it operates outside the class scope.

63. Answer: A. *
Explanation: Operators like * can be overloaded using friend functions since they do not require an object instance.
Operators like -> and ( ) cannot be overloaded using friend functions because they require access to the class's internal structure.

64. Answer: B. Encapsulation
Explanation: Encapsulation combines data and functions into a single unit (class).
It restricts direct access to data by using access specifiers like private, protecting the internal state of an object from external interference.

65. Answer: C. Private member
Explanation: Private members of a class are not accessible directly by derived classes.
- Only public and protected members are inherited.
- Protected members can be accessed within the derived class but are not accessible outside the class hierarchy.

66. Answer: C. Classes in the programming languages do not have any size
Explanation: A class itself does not occupy memory.
- Memory is allocated only when an object of the class is created.
- The size of an object depends on the size of its non-static data members, as static members are shared across all objects.

67. Answer: E. Abstract class
Explanation: Abstract classes are designed to provide a base for other classes and cannot be instantiated directly.
- They often contain pure virtual functions that must be implemented by derived classes.
- This design enforces a common interface for all derived classes.

68. Answer: C. Global variables
Explanation: Encapsulation requires restricting access to data to ensure control and security.
- Global variables are accessible from anywhere in the program, violating the principle of encapsulation.
- This unrestricted access can lead to unintended modifications and bugs.

69. Answer: C. A mechanism of combining more than one data member and member functions that implement on those data members into a single unit
Explanation: Encapsulation integrates data and methods into a single unit, typically a class.
- This ensures controlled access and better modularity in the program design.

70. Answer: D. class teacher { } s[];
Explanation: Arrays must have a defined size at the time of declaration.
- The statement class teacher { } s[]; is invalid as it does not specify the size of the array.

71. Answer: A. student class { };
Explanation: The syntax is incorrect because the class keyword must precede the class name.
- The correct syntax would be class student { };.

72. Answer: B. Passed as function
Explanation: Objects can be passed to functions by value, reference, or copy.
- However, an object itself cannot act as a function, as it represents an instance of a class.
- Functions are defined within a class or as global entities, while objects are specific instances created from a class.

73. Answer: C. Platform independent
Explanation:
- Platform independence is not an inherent feature of OOPS; it depends on the programming language.
- For example, Java is platform-independent due to its JVM, but C++ is not.
- Features like data binding, data hiding, and message passing are fundamental to OOPS.

74. Answer: D. More than one of the above
Explanation: If classes are not used, features like encapsulation, inheritance, and polymorphism cannot be implemented.
- OOPS relies on the concept of classes to organize data and behavior.
- Without classes, the program loses the structure provided by OOPS principles.

75. Answer: B. Message passing
Explanation: Message passing is the process by which objects communicate with each other in OOPS.
- It allows objects to send and receive information, facilitating interaction.
- This concept supports modularity and reusability in object-oriented design.

76. Answer: D. More than one of the above
Explanation: Abstraction in OOPS focuses on hiding the complex implementation details while exposing only the essential features or functionalities to the user. This helps reduce complexity and enhances code usability. For example, a car's dashboard provides an interface for the driver, hiding the underlying engine mechanism. This concept allows developers to design clean interfaces and modular systems, improving maintainability and reducing dependencies. Abstraction is achieved using abstract classes and interfaces in programming.

77. Answer: B. Object
Explanation: An object in OOPS is a combination of data and methods (code) that operate on the data. It is an abstraction that represents real-world entities in programming. Objects encapsulate data members (attributes) and member functions (methods) into a single unit, ensuring data security and efficient management. This concept allows programmers to create reusable and modular components, making the software more robust. Objects are instances of classes, and their behavior is defined by the class they belong to.

78. Answer: B. Is used to avoid duplication
Explanation: Abstraction helps avoid code duplication by creating a clear separation between what a system does and how it does it. By defining abstract interfaces, developers can reuse functionality without rewriting code. For instance, a single abstract class can provide a blueprint for multiple implementations, reducing redundancy. This principle improves code maintainability, scalability, and readability. It also simplifies debugging and testing by focusing on interfaces rather than internal implementations.

79. Answer: C. The data type of the data member can be easily modified without modifying any other code
Explanation: Encapsulation ensures that data members of a class are accessed and modified only through member functions, providing a controlled interface. This allows developers to change the internal data type or structure without affecting external code. For example, changing an integer to a float in a private data member will not impact the code that interacts with it via getters and setters. This promotes flexibility, modularity, and security in software design.

80. Answer: A. Ensured to some extent
Explanation: Encapsulation enhances data security by restricting direct access to the internal state of an object. It allows data to be accessed and modified only through well-defined interfaces, such as getters and setters. However, it does not provide absolute security, as data may still be exposed or manipulated if the access methods are poorly designed. Encapsulation works best when combined with other security measures, such as input validation and access control.

81. Answer: A. The above program violates the feature of encapsulation
Explanation: The program violates encapsulation by exposing the internal state (subjectmarks) of the marksofstudent class through a public method. Encapsulation requires that internal details of a class remain hidden from the outside world. Allowing direct access to private members, even indirectly, undermines the principle of data security and controlled access. This can lead to unintended modifications and bugs.

82. Answer: A. Private access specifier
Explanation: In C++, if no access specifier is explicitly mentioned, the default access level for class members is private. This ensures that data members and member functions are not accessible outside the class unless explicitly allowed. The private specifier promotes data hiding and enhances security, aligning with the principles of encapsulation. This default behavior does not apply to structs, where members are public by default.

83. Answer: A. This class can access and manipulate all the private members of the class, of which it is a friend
Explanation: A friend class in C++ has access to all private and protected members of the class that declares it as a friend. This feature allows specific classes to share implementation details without breaking encapsulation for other classes. However, it should be used sparingly, as it can increase coupling and reduce modularity. Friend classes are declared using the friend keyword.

84. Answer: C. It is the ability to process the message or data in more than one form
Explanation: Polymorphism allows objects to take on multiple forms, enabling a single interface to represent different types of behavior. It is implemented through method overloading (compile-time polymorphism) and method overriding (runtime polymorphism). For example, a function calculate() may perform differently based on the type of object invoking it. Polymorphism enhances flexibility and reusability in OOPS.

85. Answer: C. The student_details, topper, and average classes together can show the concept of polymorphism
Explanation: Polymorphism is demonstrated when a base class (student_details) provides a common interface, and derived classes (topper and average) implement the interface differently. In this example, the calculate_grade() method is overridden in each derived class, showing different behaviors for the same method. This is runtime polymorphism, achieved through method overriding.

86. Answer: C. Function is small and we want to avoid function call overhead.
Explanation: Inline functions are primarily used to reduce function call overhead for small functions. They are substituted directly into the calling code at compile time. For large functions, inlining increases the code size, leading to potential cache inefficiency. Similarly, functions with static variables are not ideal for inlining due to their state retention. Inline functions are a better alternative to macros because they provide type checking and avoid macro pitfalls like multiple evaluations of expressions.

87. Answer: A. 1
Explanation: The :: operator is the scope resolution operator, which refers to the global variable x in this case. Even though there are local variables with the same name (x) inside the function and block, the scope resolution operator ensures the global x is accessed, which has a value of 1.

88. Answer: D (B and C)
Explanation: To ensure objects can only be created using the new operator, both the constructor and destructor must be private. This prevents stack allocation and ensures controlled destruction. A friend function can be used to handle the destruction process. For example:

```
class Test {
private:
   ~Test() {}
   Test() {}
   friend void destructTest(Test*);
};
void destructTest(Test* ptr) { delete ptr; }
```

89. Answer: E. None of the above.
Explanation: In this program, no destructor is called because v1 and v2 are dynamically allocated (new) and never explicitly deleted. The destructor of a is not invoked for v1 because it contains pointers, not objects. Similarly, the destructor of v2 is not called because the program ends without deallocating v2.

90. Answer: C. 0
Explanation: In C++, all uninitialized global variables (including arrays) are automatically initialized to 0. The array x is global, so all its elements, including x[99], are initialized to 0.

91. Answer: A. The class of which it is a member.
Explanation: A member function can access all data members of its class, regardless of their access specifier (private, protected, or public). This is because it operates in the context of the class and has full access to its members.

92. Answer: D (A and B)
Explanation: Virtual functions do not need to be in the public section; they can be private or protected. Virtual functions cannot be static because static functions are resolved at compile time, while virtual functions rely on runtime polymorphism. Virtual functions are typically accessed using pointers or references to achieve polymorphism.

93. Answer: D (A, B, and C)
Explanation: Polymorphism in C++ requires inheritance (to establish a class hierarchy), virtual functions (to enable runtime polymorphism), and references or pointers (to achieve dynamic binding). Without any of these components, polymorphism cannot be implemented effectively.

94. Answer: B. The start of the method declaration.
Explanation: In C++, the final keyword can be used at the start of a method declaration to prevent it from being overridden in derived classes. For example:

```
class Base {
   virtual void display() final;
};
```

95. Answer: B. Polymorphism
Explanation: Function overloading and default arguments in C++ allow a single function name to operate differently based on parameters. This supports compile-time (static) polymorphism, where the function to execute is determined at compile time.

Inheritance is not directly supported by these features. Encapsulation deals with hiding internal details, which is unrelated to function overloading or default arguments.
Polymorphism is about having multiple forms, which is achieved here by allowing functions with the same name but different parameter lists.
Thus, the correct answer is Polymorphism.

96. Answer: D. All except 2 and 4
Explanation: Overloading based on return type alone is not allowed because the compiler cannot distinguish functions based solely on return type during a function call.
Overloading functions based on the static keyword in the return type is also not allowed, as static applies to the linkage and not the signature.
A pointer and an array are treated the same by the compiler, so this does not count as valid overloading.
Functions differing only in default arguments are not allowed because the compiler cannot resolve which version to call.
Hence, the correct answer is D. All except 2 and 4.

97. Answer: B. 5
Explanation: The function fun is declared with default arguments int fun(int = 0, int = 0). This means if no values are provided for the parameters, they default to 0.
In the main function, fun(5) is called. Here, x is assigned the value 5, while y takes its default value of 0. The function returns x + y, which is 5 + 0 = 5.

99. Answer: B. fun() fun() const
Explanation: The class Test has two overloaded fun methods. One is marked const, and the other is not. The const version is called when invoked on a const object, while the non-const version is called for non-const objects. For t1, the non-const method fun() is called. For t2, the const method fun() const is called.

100. Answer: B. Virtual functions can be static
Explanation: Virtual functions cannot be declared as static. Static functions are not associated with any object, while virtual functions rely on runtime polymorphism, which requires an object.
Virtual functions can be declared in any section (public, private, or protected) of a class.
Virtual functions should ideally be accessed using pointers to achieve runtime polymorphism.
A virtual function must be defined in the base class, although it can be overridden in derived classes.